

Projet Pédagogique

ESGI, 2026, 5ème année Ingénierie du Web (Janvier)

Contexte

Vous êtes développeur·se full-stack au sein d'une startup qui vient d'être financée. Le produit à développer est une application web nommée **ConsignArt** — une plateforme B2B dédiée aux galeries d'art contemporain pour gérer la **consignation d'œuvres d'art**.

Le métier de la consignation est spécifique : un artiste confie une ou plusieurs œuvres à une galerie, qui les expose et les vend pour son compte. La galerie touche une commission sur chaque vente, et doit reverser le solde à l'artiste. Chaque œuvre peut changer de statut (disponible, en exposition, vendue, rendue). Les flux financiers doivent être tracés avec précision.

Objectif pédagogique : ce projet doit couvrir l'ensemble du socle technique de Nest.js — modules, contrôleurs, services, providers, DTOs, pipes, guards, interceptors, exceptions filters, TypeORM, transactions, validation, tests et conteneurisation Docker.

Règles générales

- **Langue du code** : tout le code source, les messages de commit, la documentation et le nommage sont en **anglais** (sauf le présent sujet).
- **Stack imposée** : Nest.js côté backend, avec TypeORM et SQLite (ou PostgreSQL via Docker). Base de données au choix, justifiée en soutenance.
- **Frontend** : non exigé. L'API REST est le livrable principal. Une interface minimale (Swagger ou une surcouche React/Vue optionnelle) est un bonus.
- **Docker** : obligatoire. Le projet doit pouvoir être lancé avec un seul `docker compose up --build`.
- **Tests** : des tests unitaires (Vitest) et/ou d'intégration sont attendus sur les parties critiques (services, guards, pipes).
- **Rendu** : un dépôt Git (GitHub, GitLab, etc.) avec un historique de commits propre et des messages explicites.
- **Soutenance** : 20 minutes de démo + 10 minutes de questions. La note évalue la **qualité du code**, la **couverture des concepts Nest.js**, le **raisonnement métier** et

Cahier des charges fonctionnel

1. Gestion des utilisateurs et authentification

Le système distingue quatre rôles :

Rôle	Description
admin	Gère la plateforme (validation des comptes, statistiques)
gallery	Galerie d'art : dépose et gère des œuvres en consignment
artist	Artiste : suit ses œuvres et consulte ses revenus
collector	Collectionneur : consulte le catalogue et achète

Chaque utilisateur s'inscrit, se connecte et accède à des ressources selon son rôle. L'authentification se fait par **JWT** (access token + refresh token).

Règles métier :

- Un artiste ne peut être rattaché qu'à une seule galerie à la fois.
- Un compte `gallery` doit être validé par un `admin` avant d'être actif.
- Les mots de passe sont hachés avec `bcrypt`.

2. Gestion des artistes

Une galerie peut enregistrer des artistes dans son catalogue. Chaque artiste a :

- Un nom, un prénom, une biographie, un portfolio (URL), une nationalité.
- Un statut (`active`, `inactive`).
- Une date d'entrée dans la galerie.

Un artiste peut être transféré d'une galerie à une autre (avec l'accord de l'admin).

3. Gestion des œuvres

Une œuvre appartient à un artiste et est déposée dans une galerie. Attributs :

- Titre, description, année de création, technique (huile, photographie, sculpture, etc.)
- Dimensions (hauteur, largeur, profondeur — optionnelle selon la technique)
- Prix de vente (en euros)

- Prix de réserve (prix plancher en dessous duquel la galerie ne peut pas vendre)
- Statut : `available` , `on_loan` (prêtée à une autre exposition), `sold` , `returned`
- Image (URL)
- Date de dépôt en consignment

Règles métier :

- Une œuvre ne peut être vendue en dessous de son prix de réserve.
- Une œuvre au statut `on_loan` ne peut pas être vendue.
- Le changement de statut est tracé (on garde l'historique).
- Un artiste ne peut pas avoir plus de 50 œuvres actives simultanément dans une galerie.

4. Ventes et commissions

Quand une œuvre est vendue à un collectionneur :

1. Un **contrat de vente** est créé (acheteur, œuvre, prix de vente, date).
2. La **commission** de la galerie est calculée selon la règle :
 - 40 % si le prix de vente $\leq$ 5 000 €
 - 35 % si 5 000 € < prix $\leq$ 20 000 €
 - 30 % si prix > 20 000 €
3. Le **solde artiste** (prix de vente – commission) est enregistré.
4. Une facture est générée pour l'acheteur et un relevé de vente pour l'artiste (simples enregistrements en base).

Un historique des ventes est consultable par la galerie, l'artiste et l'admin.

5. Exposition et prêts

Une galerie peut organiser des **expositions** (physiques ou virtuelles). Une exposition a :

- Un nom, une date de début, une date de fin, un lieu (ou un lien virtuel).
- Une liste d'œuvres sélectionnées.

Pendant la durée de l'exposition, les œuvres concernées passent en statut `on_loan` .

Une œuvre peut être **prêtée** à une autre galerie pour une exposition temporaire. Le prêt est enregistré avec les dates et les conditions.

Règles métier :

- On ne peut pas prêter une œuvre déjà prêtée.

- Une œuvre en exposition ne peut pas être vendue (elle est `on_loan`).
- Une exposition ne peut pas être créée avec zéro œuvre.

6. Rapports et tableau de bord

L'API expose des endpoints de statistiques :

- **Pour une galerie** : nombre d'œuvres vendues par mois, chiffre d'affaires total, top 5 des artistes les plus vendus, taux de rotation des œuvres.
 - **Pour un artiste** : total des ventes, commissions versées, œuvres encore disponibles.
 - **Pour l'admin** : nombre d'utilisateurs actifs, volume de transactions, commissions totales perçues par la plateforme.
-

Contraintes techniques (Nest.js)

Le projet doit impérativement illustrer les concepts suivants de Nest.js :

Modules

- Découpage fonctionnel en modules (au moins : `auth`, `users`, `artists`, `artworks`, `sales`, `exhibitions`, `reports`).
- Chaque module encapsule son domaine (contrôleurs, services, entités, DTOs).

Contrôleurs et routes

- Routes REST respectant les conventions (noms au pluriel, verbes HTTP, statuts codes appropriés).
- Paramètres de route, query string, body correctement typés.
- Versionnement de l'API via préfixe (`/api/v1/...`).

Pipes

- **Validation automatique** des DTOs avec `class-validator` et `class-transformer` (utilisation de `ValidationPipe` global).
- **Pipe personnalisé** : au moins un pipe de transformation métier (ex. : parsing d'une date au format français, normalisation d'un prix, transformation d'un statut).
- **Pipe personnalisé** : au moins un pipe de validation métier (ex. : vérifier qu'un artiste n'a pas déjà 50 œuvres, ou qu'une œuvre n'est pas déjà vendue).

Guards

- **Guard d'authentification** global (`JwtAuthGuard`) qui vérifie la validité du token JWT.
- **Guard de rôle** personnalisé (`RolesGuard`) avec décorateur `@Roles()` pour restreindre l'accès selon le rôle utilisateur.
- **Guard d'appartenance** personnalisé (ex. : `OwnershipGuard`) qui vérifie qu'une œuvre appartient bien à la galerie de l'utilisateur connecté.

Interceptors

- **Interceptor de transformation** : formate toutes les réponses API dans un enveloppe standard (`{ data, meta, timestamp }`).
- **Interceptor de logging** : enregistre chaque requête (méthode, route, durée, utilisateur) dans un fichier ou en base.
- **Interceptor de cache** (optionnel) : met en cache les réponses des endpoints de consultation publique.

Exception Filters

- **Filtre global** qui attrape toutes les exceptions non gérées et retourne une réponse formatée.
- **Filtre personnalisé** pour les erreurs métier spécifiques (ex. `BusinessRuleViolation`).

TypeORM et base de données

- Utilisation de **TypeORM** avec une base de données SQL (SQLite en dev, PostgreSQL en prod via Docker).
- **Relations** : `ManyToOne` , `OneToMany` , `ManyToMany` (ex. : œuvre ↔ exposition).
- **Transactions** : une opération critique (ex. : vente d'une œuvre avec mise à jour du solde artiste) doit se faire dans une transaction.
- **Migrations** : le projet doit inclure des migrations pour créer le schéma de base.
- **Index** : au moins un index sur une colonne fréquemment interrogée (ex. : `status` d'une œuvre).

Tests

- Tests unitaires sur les **services** critiques (vente, commission, changement de statut).
- Tests unitaires sur les **guards** et **pipes** personnalisés.
- Tests d'intégration sur au moins un **endpoint** complet.
- Utilisation de **Vitest** (ou Jest).

Configuration et environnement

- Variables d'environnement chargées via `@nestjs/config`.
 - Fichier `.env.example` fourni sans secrets.
 - Configuration typée avec `ConfigService` et validation des variables requises (avec `Joi` ou `class-validator`).
-

Docker

Le projet doit être conteneurisé :

```
# Structure attendue du docker-compose.yml
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
      - DB_PORT=5432
      - DB_USERNAME=...
      - DB_PASSWORD=...
      - JWT_SECRET=...
    depends_on:
      - db
  db:
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: ...
      POSTGRES_PASSWORD: ...
      POSTGRES_DB: consignant

volumes:
  pgdata:
```

- Un `Dockerfile` multi-stage (build + production) avec une image finale légère (`node:20-alpine`).
- `docker compose up --build` doit tout lancer.
- Les données doivent être persistantes via un volume Docker.

Livrables

1. **Dépôt Git** avec historique propre et messages de commit conventionnels (conventional commits).
2. **Code source complet** de l'API Nest.js.
3. **Fichiers Docker** (Dockerfile, docker-compose.yml, .dockerignore).
4. **Fichier de configuration** (.env.example).
5. **Documentation** minimale dans un `README.md` :
 - Description du projet
 - Instructions de lancement (`docker compose up --build`)
 - Liste des fonctionnalités implémentées
 - Choix techniques justifiés
 - Diagramme (optionnel mais apprécié)

Barème de notation (soutenance — 20 points)

Critère	Détail
Présentation orale	Clarté de l'exposé, démonstration en direct, capacité à expliquer les choix
Qualité du code	Architecture, découpage en modules, lisibilité, respect des principes SOLID
Couverture Nest.js	Utilisation pertinente des contrôleurs, services, pipes, guards, interceptors, filters, TypeORM
Fonctionnalités métier	Complétude et exactitude des règles de gestion implémentées
Docker & déploiement	Dockerfile, docker-compose fonctionnel, image légère
Tests	Présence et pertinence des tests unitaires et/ou d'intégration
Questions	Les réponses aux questions peuvent impacter les autres critères (bonus/malus)

Note éliminatoire : un projet qui ne démarre pas avec `docker compose up`, qui ne contient aucun guard/pipe personnalisé, ou dont le code généré par l'IA n'est

pas compris, expliqué ou maîtrisé par l'étudiant·e lors de la soutenance, sera automatiquement en dessous de 10/20.